

Formal Yöntemler: Yazılım ve Donanım Doğruluğunun Matematiksel Temelleri

Ömer Faruk Yavuz

May 2026

Formal Yöntemler: Yazılım ve Donanım Doğruluğunun Matematiksel Temelleri

Tanım ve Temel Yaklaşım

Formal yöntemler (*formal methods*), yazılım ve donanım sistemlerinin doğruluğunu deneysel test yerine matematiksel ispat yoluyla göstermeye çalışan disiplini ifade eder. Geleneksel yazılım geliştirme pratiğinde sistem kodlanır, çeşitli test senaryolarıyla denenir ve hatalı bir davranış gözlemlenmediği sürece kabul edilebilir sayılır. Bu yaklaşımın temel epistemolojik kısıtı şudur: test edilen vakalar başarıyla geçilse dahi, test edilmemiş vakalar hakkında hiçbir bilgi elde edilmez. Söz konusu sınırlılık, daha önce ele alınan davranış uzayı kavramı çerçevesinde şöyle ifade edilebilir: test, davranış uzayının yalnızca küçük bir bölgesini ziyaret eder ve geri kalan bölge için herhangi bir garanti üretmez.

Formal yöntemler, bu epistemolojik açığı kapatma iddiasını taşır. Yaklaşım üç temel hamleden oluşur: sistem matematiksel olarak tanımlanır; sistemin sağlaması beklenen kurallar (değişmezler) matematiksel olarak formüle edilir; ardından otomatik veya yarı-otomatik bir mekanizma, söz konusu kuralların tüm olası durumlarda sağlandığını ispatlar. Elde edilen sonuç deneysel bir doğrulama değil, davranış uzayının tamamı üzerinde geçerli olan matematiksel bir ifadedir.

Temel Bileşenler

Bir formal yöntem altyapısı üç bileşen üzerine kurulur.

Spesifikasyon Dili

Sistemin matematiksel olarak ifade edilebilmesini sağlayan dildir. Bu kategoride çeşitli diller geliştirilmiştir; başlıcaları TLA+ (Leslie Lamport), Alloy (Daniel Jackson), Z notation (Jean-Raymond Abrial ve ardından J.M. Spivey), Coq, Lean, Isabelle, Dafny, P ve F* dilleridir. Her dilin temel felsefesi farklı olmakla birlikte, hepsi sembolik mantık ve küme teorisi üzerine inşa edilmiştir ve sistemin durum geçişlerini, eylemlerini ve sınırlarını ifade etmeye olanak tanır.

Özellik Dili

Sistemin asla ihlal etmemesi gereken kuralları (invariants) ya da eninde sonunda sağlaması gereken özellikleri (liveness properties) ifade etmek için kullanılan dildir. Söz konusu diller genellikle birinci dereceden mantığa zamansal işleçler eklenmiş hâliyle (temporal logic) tanımlanır; zira yazılım sistemleri zaman içinde durum değiştirir ve birçok özellik “her zaman”, “eninde sonunda” veya “şu olduğunda mutlaka şu olmalı” biçiminde ifade edilmeyi gerektirir.

Doğrulama Mekanizması

Tanımlanan sistemin, yazılan özellikleri ihlal edip etmediğini denetleyen bileşendir. TLC (TLA+ için), SPIN ve NuSMV gibi araçlar bu kategoride yer alır. Doğrulama mekanizmaları iki temel yaklaşımdan birini benimser.

İki Ana Doğrulama Stratejisi

Model Denetleme

Model denetleme (*model checking*), sonlu sayıda duruma sahip sistemlerin tüm olası yürütmelerinin sistematik biçimde taranması ilkesine dayanır. Yaklaşım otomatiktir; geliştiricinin yapması gereken tek şey sistemi ve özellikleri yazmaktır. Durum uzayı sonsuz olduğunda yöntem doğrudan uygulanamaz; bu durumda soyutlama (abstraction) ya da sınırlandırma (bounded model checking) teknikleri devreye girer. Yöntemin pratikteki en güçlü yanı, bir özellik ihlali tespit edildiğinde somut bir karşı örnek (counter-example) sunmasıdır: “şu sırayla şu eylemleri gerçekleştiren bir yürütme, ilgili değişmezi ihlal eder.” Söz konusu somut izler, mühendisin sorunu doğrudan kavramasına ve düzeltmesine olanak tanır.

Teorem İspatı

Teorem ispatı, sistemin doğruluğunun matematiksel bir kanıt inşa edilerek gösterilmesi yaklaşımıdır. Geliştirici bir teorem yazar (“bu sistem şu özelliği sağlar”) ve adım adım bir ispat üretir; makine ise her ispat adımının geçerliliğini doğrular. Coq, Isabelle/HOL ve Lean bu kategorideki başlıca araçlardır. Yöntem sonsuz durum uzaylarına da uygulanabilir, ancak insan emeği maliyeti çok yüksektir. Bir karşılaştırma olarak, formel olarak doğrulanmış seL4 mikroçekirdeği yaklaşık yirmi beş kişi-yılı insan emeği gerektirmiştir.

Formal Yöntemlerin Spektrumu

Formal yöntemler tek bir maliyet ve fayda seviyesinde değildir; bir spektrum oluştururlar.

Ağır Formal Yöntemler

Coq, Isabelle veya Lean ile yapılan tam teorem ispatı bu kategoriye girer. Sertifikalı C derleyicisi CompCert ve formel olarak doğrulanmış mikroçekirdek seL4 örnek olarak gösterilebilir. Bu seviyedeki projeler genellikle on milyonlarca dolar büyüklüğünde olup yıllar süren bir ekip çalışması gerektirir.

Orta Düzey Formal Yöntemler

TLA+ ve PlusCal ile yapılan model denetleme, sözleşmeye dayalı tasarım (*design-by-contract*, ilk olarak Bertrand Meyer'in Eiffel dilinde sistematikleştirilmiştir) ve arıtım tipleri (refinement types) bu kategoride yer alır. AWS'nin iç pratiklerinde yaygın olarak kullanılan seviye budur.

Hafif Formal Yöntemler

Endüstride “hafif formal yöntemler” (*lightweight formal methods*) olarak adlandırılan bu kategori, formel düşüncenin günlük geliştirme pratiğine taşınmış hâlidir. Güçlü tip sistemleri (Rust'ın ödünç alma denetleyicisi, Haskell'in tip sistemi, TypeScript), özellik tabanlı test (*property-based testing*: QuickCheck, Hypothesis) ve statik analiz araçları (CodeQL, Semgrep) bu kategoride yer alır. Söz konusu araçlar tam bir ispat sunmaz, ancak belirli hata kategorilerini matematiksel kesinliğe yakın garantilerle eler. Çoğu modern geliştirici, farkında olmaksızın bu seviyede formal yöntemlerle çalışmaktadır.

Tarihsel Gelişim

Formal yöntemlerin temelleri, 1960'ların sonu ile 1970'lerin başında atılmıştır. Tony Hoare'ın Hoare mantığı ve Edsger Dijkstra'nın en zayıf önkoşul (weakest precondition) kavramı, programların matematiksel olarak ispatlanabileceği fikrinin ilk sistematik ifadeleridir. Söz konusu dönem akademik bir hareket olarak başlamış ve endüstriyel uygulamadan uzak kalmıştır.

1980'lerde önemli diller geliştirilmiştir: Leslie Lamport TLA'yı, Tony Hoare CSP'yi ve J.M. Spivey Z notation'ı bu dönemde formüle etmiştir. Aynı dönemde model denetleme, Edmund Clarke ve E. Allen Emerson tarafından icat edilmiş; bu çalışmaları nedeniyle 2007 yılında Joseph Sifakis ile birlikte Turing Ödülü'nü almışlardır.

1990'larda araçlar olgunlaşmış; SPIN ve NuSMV gibi model denetleyiciler endüstride ilk ciddi uygulamalara konu olmuştur. Intel'in 1994 yılındaki Pentium FDIV hatası (yaklaşık 475 milyon dolarlık bir geri çağırma maliyetiyle sonuçlanmıştır) sonrasında şirket, çip tasarımı formel doğrulamayı rutin bir prosedüre dönüştürmüştür.

2000'lerde Coq ve Isabelle olgunluğa erişmiş; Microsoft Research'ün SLAM projesi, Windows aygıt sürücülerini statik olarak doğrulayarak milyonlarca hatayı tespit etmiştir.

2010'larda AWS'nin 2011 itibarıyla TLA+'ı benimsemesi, endüstri için bir dönüm noktası olmuştur. Aynı dönemde FoundationDB ekibi, deterministik

simülasyon testi (*deterministic simulation testing*) yaklaşımıyla yeni bir alt disiplin başlatmıştır.

2020’lerde Lean matematik camiasında popülerleşmiş; Dafny ve P dili programcı topluluğuna daha yakın bir arayüz sunmuştur. Hafif formal yöntemler terimi yaygınlaşmış; Marc Brooker’ın 2024 yılında *ACM Queue* dergisinde yayımladığı “Systems Correctness Practices at AWS” makalesi, bu hareketin programatik bir manifestosu niteliği taşımaktadır.

Endüstriyel Uygulamalar

Formel yöntemler bugün birçok farklı sektörde rutin kullanım bulmaktadır.

Kuruluş	Araç / Yaklaşım	Uygulama Alanı
Amazon Web Services Microsoft	TLA+, P, Dafny, Kani SDV, TLA+	S3, DynamoDB, EBS, IAM Windows sürücüleri, Azure Cosmos DB
Intel	Çeşitli	Çip tasarımı, önbellek tutarlılığı
NASA	SPIN	Mars gezgini, kritik aviyonik
Airbus	SCADE	A380 fly-by-wire kontrol yazılımı
NICTA / seL4 ekibi	Isabelle/HOL	Mikroçekirdek formel ispatı
Jane Street	OCaml + QuickCheck	Finansal işlem kodları
FoundationDB, TigerBee- tle, Antithesis	Deterministik simülasyon	Veritabanı doğruluğu

Söz konusu liste, formel yöntemlerin tamamen niş bir alan olmadığını, ancak hâlâ özel ve sınırlı bir uzmanlık alanı olduğunu göstermektedir.

Yaygın Benimsenmenin Önündeki Engeller

Formel yöntemlerin yaygın endüstriyel benimsenmesinin önünde dört yapısal engel bulunur.

İlki, öğrenme eğrisinin dikliğidir. Yöntem; sembolik mantık, küme teorisi ve zamansal mantık (temporal logic) bilgisi gerektirir. Söz konusu birikim, yazılım mühendisliği eğitiminin standart müfredatında yer almaz.

İkincisi, spesifikasyon yazma maliyetidir. Bir özelliğin TLA+ veya benzeri bir dilde modellenmesi, aynı özelliğin doğrudan kodlanmasından üç ila beş kat daha fazla zaman gerektirebilir.

Üçüncüsü, fayda-maliyet dengesidir. Çoğu yazılım projesinde hata maliyeti, formel doğrulamanın getirdiği yatırım maliyetini haklı kılacak düzeyde değildir. Yöntem, hata maliyetinin çok yüksek olduğu kritik sistemlerde anlamlı hâle gelir: havacılık, finansal işlem altyapıları, dağıtık veritabanları ve kriptografik protokoller bu alanların başında gelir.

Dördüncüsü, eğitim altyapısının yetersizliğidir. Üniversitelerin bilgisayar bilimleri programlarının büyük çoğunluğu formal yöntemleri zorunlu ders olarak içermez; konu ya seçmeli bırakılır ya da yüksek lisans seviyesine ertelenir.

Hafif Formal Yöntemlerin Ana Akıma Sızması

Formal yöntemler dar anlamıyla yaygın olmasa da, hafif versiyonları günümüz yazılım pratiğine derinlemesine nüfuz etmiş durumdadır. TypeScript ile yazılan her satır, tip sisteminin sağladığı küçük bir ispattan geçer. Rust'ın ödünç alma denetleyicisi, bellek güvenliğini derleme zamanında formel olarak garanti eder. Bir `assert` ifadesi, çalışma zamanında bir değişmezin denetimidir ve formel

düşüncenin günlük bir tezahürüdür. Özellik tabanlı test ise, küçük ölçekli bir formel doğrulama deneyi olarak değerlendirilebilir.

Söz konusu nüfuz, sektörün farkında olmaksızın formel disipline doğru kaymakta olduğunu gösterir. Hafif araçlar, ağır formal yöntemlerin “doğruluğu garanti etme” misyonunun bir alt kümesini, çok daha düşük bir maliyetle ve dik bir öğrenme eğrisi olmaksızın yerine getirir.

Sonuç

Formel yöntemler, doğruluğu deneysel testten matematiksel ispata taşıyan bir paradigma kaymasını temsil eder. Yaklaşım pahalıdır ve çoğu yazılım projesinde aşırı bir yatırım sayılır; ancak davranış uzayı geniş ve hata maliyeti yüksek olan sistemlerde, mevcut yöntemler arasında en güçlü doğruluk garantisini sunar. Tarihsel olarak altmış yıllık bir akademik birikime sahip olmakla birlikte, endüstriyel olgunluğu yalnızca son on yıllarda belirgin hâle gelmiştir. Hafif versiyonlarının modern dillerin tip sistemleri ve test çerçeveleri içinde yaygınlaşması, sektörün uzun vadeli yöneliminin matematiksel doğruluk garantilerine doğru kaydığını göstermektedir.